

MODULE PRACTICAL DATA SCIENCE (WS25-26)



Capstone Project

Predicting Career Domain and Seniority Using LinkedIn Profiles from the SnapAddy Dataset

Group members:

Lennart Redlich - 3270019

Anh Pham - 3142126

Van Anh Nguyen - 3151619

Würzburg, Jan 2026

Table of Contents

1. Executive Summary	3
2. Problem Statement & Objectives	4
3. The Finished Product: Streamlit Dashboard	5
3.1. Key Features.....	5
3.2. Use Cases.....	5
3.3. Technical Architecture	5
4. Data & Exploratory Data Analysis	7
4.1. Dataset provided by SnapAddy	7
4.2. Annotated LinkedIn CV Dataset.....	7
4.3. Not Annotated LinkedIn CV Dataset.....	10
4.4. External Label Datasets (Department-v2 & Seniority-v2).....	11
5. Modeling Approaches.....	12
5.1 Baseline: Rule-Based Lookup Approach	12
5.2 Embedding-based Seniority Classification (Zero-shot).....	14
5.2.1 Embedding-based (Zero-shot) Seniority Classification.....	14
5.2.2 Embedding-based (Zero-shot) Department Classification.....	15
5.3 TF-IDF + Logistic Regression.....	18
5.3.1 TF-IDF + Logistic Regression Seniority	18
5.3.2 TF-IDF + Logistic Regression Department	20
5.4. Features Engineering and Conventional Models	22
5.4.1 Department Classification.....	22
5.4.2 Seniority Classification.....	23
5.5. Fine-tuning: Domain & Seniority	25
5.6 Pseudo-Labeling & Self-Training (TF-IDF).....	27
5.7 LLM-Based Classification (Google Gemini API).....	30
5.7.1 Seniority	30
5.7.2 Department.....	32
6. Performance Comparison	36
7. Model Failures & Limitations.....	37
7.1 Seniority Prediction Challenges.....	37
7.2 Department Prediction Challenges	37
8. Generative AI Usage Documentation.....	38
8.1 Tools Used.....	38

8.2 Use Cases & Examples.....	38
8.2.1 Code Generation & Debugging	38
8.2.2 API Integration.....	38
8.2.3 Data Analysis	38
8.2.4 Documentation & Report Writing	38
8.3 Reflection on AI Assistance.....	38
9. Conclusion	40
10. Role of each Group Members	41
11. Appendix.....	42
11.1 Code Structure	42
11.2 Required Python Packages.....	42
11.3 GitHub Repository.....	43

1. Executive Summary

This project develops an end-to-end machine learning pipeline to automatically predict two key professional characteristics from LinkedIn profiles: (1) professional domain (e.g., Sales, IT, Marketing) and (2) seniority level (e.g., Junior, Senior, Director). The solution addresses SnapAddy's need for automated classification of career profiles.

We implemented and evaluated four distinct modeling approaches: rule-based baseline, TF-IDF with logistic regression, embedding-based classification, and LLM-powered API predictions. A Streamlit dashboard demonstrates the practical application, enabling real-world use cases such as candidate filtering and targeted outreach.

Key Results:

- Seniority prediction: 60% exact accuracy with TF-IDF (82% within ± 1 level)
- Department prediction: 48% accuracy with LLM API
- Functional web application with real-time predictions using Streamlit

2. Problem Statement & Objectives

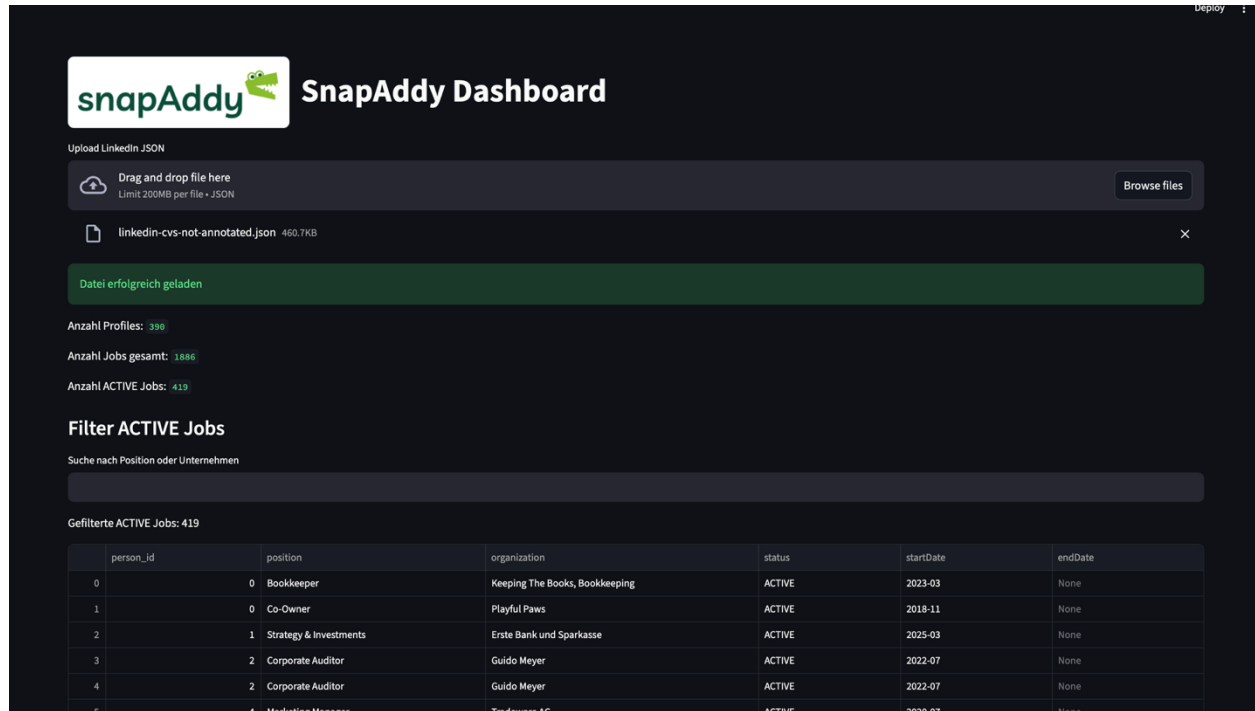
SnapAddy requires automated classification of LinkedIn career profiles to support business development and recruiting operations. Manual classification is time-consuming and inconsistent. The goal is to predict:

- **Professional Domain:** 10 categories including Sales, IT, Marketing, Consulting, HR, Administrative, Business Development, Customer Support, Project Management, and Purchasing
- **Seniority Level:** 6 hierarchical levels from Junior to Director

Success Criteria:

- Implement multiple modeling approaches
- Evaluate on hand-labeled SnapAddy dataset
- Deploy functional prototype application (Streamlit Dashboard)

3. The Finished Product: Streamlit Dashboard



We developed an interactive web application using Streamlit that demonstrates the practical application of our approaches. The dashboard serves as both a proof-of-concept and a tool for business users.

3.1. Key Features

- **Data Input:** Load LinkedIn CV data from JSON files
- **Search & Filter:** Text search across job titles and organizations, filter by predicted seniority and department
- **Real-time Predictions:**
Seniority is predicted using a locally deployed TF-IDF + Logistic Regression model.
Department is inferred using our API-based LLM classification approach
- **Interactive Results:** Display filtered results in sortable tables with relevant metadata

3.2. Use Cases

- **Recruiting:** Find candidates with specific seniority levels in target departments
- **Sales Targeting:** Identify decision-makers (e.g., Director/Management level) in specific functions
- **Lead Scoring:** Prioritize outreach based on role characteristics

3.3. Technical Architecture

The application combines two prediction components within a Python/Streamlit frontend:

- A pre-trained TF-IDF + Logistic Regression model (stored as .joblib) for **seniority prediction**. The model is loaded on startup and used for on-demand inference during filtering and exploration.
- An API-based LLM inference pipeline for department classification, where the job title and organization name are jointly provided as input to an external large language model (Google Gemini). The model is prompted to select exactly one department label from a predefined set of project-specific classes, ensuring consistency with the ground-truth labeling scheme. The returned textual label is then parsed and mapped directly to the corresponding department used in the evaluation and filtering logic.

Code Reference: app.py

4. Data & Exploratory Data Analysis

4.1. Dataset provided by SnapAddy

SnapAddy provided a set of structured datasets derived from LinkedIn CVs to support the development and evaluation of the proposed classification approaches. The data consists of two LinkedIn CV datasets in JSON format and two accompanying CSV files defining the target label spaces for department and seniority prediction.

The first JSON dataset contains annotated LinkedIn CVs, where each CV is represented a list of job entries. Each job entry includes information such as job title, organization name, employment dates, and employment status (ACTIVE or INACTIVE), as well as manually assigned labels for department and seniority. This dataset serves as the primary ground truth for model development and evaluation.

The second JSON dataset contains unannotated LinkedIn CVs following the same structural schema but without department or seniority labels. This dataset is used exclusively for inference and demonstration purpose, including the deployment of the Streamlit application.

In addition to the CV data, SnapAddy provided two CSV files defining the department and seniority taxonomies used throughout the project. These files specify the allowed target classes and provide example job titles for each label, thereby ensuring a consistent and predefined label space across all modeling approaches.

Together, these datasets form the foundation for the exploratory data analysis, model training, evaluation, and deployment stages of the project.

4.2. Annotated LinkedIn CV Dataset

The annotated LinkedIn CV dataset comprises 609 individual CVs, each represented as a list of job entries corresponding to a person's professional history. Each job entry includes structured information such as:

- organization
- linkedin
- position
- startDate
- endDate
- status
- department
- seniority

For the purpose of this project, only job entries marked with the status ACTIVE are considered, as the prediction target is explicitly defined as the individual's current professional role. Across

the dataset, this results in 623 active job entries, indicating that a small number of individuals hold more than one active position simultaneously. However, the majority of CVs contain exactly one active job, while multiple active roles are comparatively rare.

```
from collections import Counter

active_job_counts = []

for cv in cvs:
    count = sum(1 for job in cv if job.get("status") == "ACTIVE")
    active_job_counts.append(count)

Counter(active_job_counts)
```

```
Counter({1: 380, 0: 131, 2: 78, 3: 10, 5: 4, 4: 3, 8: 1, 7: 1, 10: 1})
```

What caught our eye is that 131 CVs contain zero job entries marked as ACTIVE. A closer inspection reveals that these CVs either consist exclusively of job entries labeled as INACTIVE or contain positions with an undefined employment status (“UNKNOWN”) or a combination of both (“INACTIVE” & “UNKNOWN”).

```
def cv_status_profile(cv):
    return set(job.get("status") for job in cv)

from collections import Counter
cv_status_patterns = Counter(
    tuple(sorted(cv_status_profile(cv)))
    for cv in cvs
)
```

```
cv_status_patterns
```

✓ 0.0s

```
Counter({('ACTIVE', 'INACTIVE'): 372,
        ('UNKNOWN',): 97,
        ('ACTIVE',): 89,
        ('INACTIVE',): 32,
        ('ACTIVE', 'UNKNOWN'): 13,
        ('ACTIVE', 'INACTIVE', 'UNKNOWN'): 4,
        ('INACTIVE', 'UNKNOWN'): 2})
```

Since the prediction target of this project is explicitly defined as the individual's current professional role, only job entries marked as ACTIVE are used as prediction targets. Job entries with status INACTIVE or UNKNOWN are not treated as target instances but may be leveraged as contextual information during preprocessing, for example to infer missing department or seniority information when such labels are not available for the active position.

In the following, we take a closer look at the distribution of department and seniority labels in the annotated dataset.

Both target variables, department and seniority, are fully populated in the annotated dataset, with no missing values at the job-entry level.

```
df[["department", "seniority"]].isna().mean()

✓ 0.0s

department    0.0
seniority      0.0
dtype: float64
```

The distribution of department labels reveals a pronounced class imbalance. Nearly half of all job entries (47.5%) are assigned to the category “Other”, indicating that a substantial share of roles cannot be unambiguously mapped to a specific functional domain.

```
df["department"].value_counts(normalize=True)

✓ 0.0s

department
Other                0.474602
Information Technology 0.118271
Sales                0.083017
Consulting           0.073920
Project Management   0.066338
Marketing             0.050417
Administrative        0.031842
Business Development 0.029568
Purchasing           0.027293
Human Resources       0.026535
Customer Support      0.018196
Name: proportion, dtype: float64
```

To better understand the dominance of the Other category, we inspected the most frequent job titles assigned to this label.

The category compromises a heterogeneous set of executive, ownership, and academic roles, which cannot be clearly assigned to a single functional department. This explains the high prevalence.

```
df[df["department"] == "Other"]["position"].value_counts().head(15)
```

✓ 0.0s

position	
Geschäftsführer	41
Managing Director	22
CEO	20
Owner	12
Director	11
Vice President	11
General Manager	10
Geschäftsführender Gesellschafter	9
Unternehmensinhaber	9
Rechtsanwalt	8
Professor	8
CFO	7
Research Assistant	7
Intern	7
Chief Executive Officer	6

Name: count, dtype: int64

The distribution of seniority labels is dominated by the category Professional, which accounts for approximately 46% of all job entries. This is followed by Lead (17%) and Management (16%), while Junior, Senior, and Director roles are comparatively less frequent.

```
df["seniority"].value_counts(normalize=True)
```

✓ 0.0s

seniority	
Professional	0.462092
Lead	0.174375
Management	0.158453
Junior	0.087187
Senior	0.064443
Director	0.053450

Name: proportion, dtype: float64

Code Reference: 01_EDA.py

4.3. Not Annotated LinkedIn CV Dataset

In addition to the annotated CVs, SnapAddy provided a second dataset consisting of LinkedIn CVs without department and seniority labels.

This unannotated dataset contains 390 CVs and follows the same structural schema as the annotated data, including job position titles, organization names, employment dates, and job status information.

In this project, the unannotated CVs are used to demonstrate the applicability of the developed models in a real-world setting, particularly within the Streamlit dashboard, where predictions for department and seniority are generated for previously unseen profiles.

Code Reference: 01_EDA.py

4.4. External Label Datasets (Department-v2 & Seniority-v2)

In addition to the LinkedIn CV datasets, the project also provides two external label resources that support model development and label standardization: **department-v2.csv** and **seniority-v2.csv**. These datasets contain short text samples paired with their corresponding target labels and can be used to enhance classification robustness, particularly in cases where job titles alone are ambiguous.

The **department-v2** dataset contains 11 department labels in total. The distribution is highly concentrated in a few major categories. The largest classes are Marketing (4295) and Sales (3328), followed by Information Technology and Business Development. Smaller categories include Project Management and Consulting. The remaining classes appear only rarely, such as Administrative, Other, Purchasing, Customer Support, and Human Resources. This imbalance indicates that even in the external dataset, department classification is dominated by common business functions.

```
df_department_v2_labels["label"].value_counts()
✓ 0.0s
```

label	count
Marketing	4295
Sales	3328
Information Technology	1305
Business Development	620
Project Management	201
Consulting	167
Administrative	83
Other	42
Purchasing	40
Customer Support	33
Human Resources	31

Name: count, dtype: int64

The **seniority-v2** dataset contains five seniority labels, namely Junior, Senior, Lead, Management, and Director. The class distribution is also imbalanced, with Senior (3733) and Lead (3546) forming the majority of samples. Director (984) and Management (756) occur less frequently, while Junior (409) is the smallest class. This confirms that seniority classification is skewed toward

mid- to high-level roles, and models may struggle more with underrepresented categories such as Junior.

Notably, the seniority label Professional appears in the annotated LinkedIn CV dataset but is not part of the predefined seniority taxonomy provided in seniority-v2.csv.

This discrepancy reflects differences between real-world LinkedIn annotations and the standardized label scheme and requires an explicit normalization step during preprocessing to ensure label consistency across datasets.

```
df_seniority_v2["label"].value_counts()
✓ 0.0s
```

label	
Senior	3733
Lead	3546
Director	984
Management	756
Junior	409

Name: count, dtype: int64

Code Reference: 01_EDA.py

5. Modeling Approaches

We implemented 7 approaches to address both classification tasks, ranging from simple rule-based heuristics to advanced language model APIs.

5.1 Baseline: Rule-Based Lookup Approach

Methodology:

As a conservative baseline, we implement a rule-based lookup approach for both seniority and department prediction, that assigns seniority labels based on exact string matches between job titles in the annotated LinkedIn CVs and the predefined seniority / department taxonomy (seniority-v2.csv & department-v2.csv).

For each active job position, the normalized job title is matched against the taxonomy entries. If a match is found, the corresponding seniority / department label is assigned; otherwise, no prediction is produced.

Seniority Baseline Results

For seniority prediction, the lookup baseline achieves a match rate of 22.8%, indicating that fewer than one quarter of active job titles can be matched exactly to the predefined seniority taxonomy.

On the matched cases, the approach reaches an accuracy of 79.3%, covering 121 out of 623 active job entries.

```
df_active["seniority_pred_lookup"] = df_active["position_norm"].map(lookup_sen)

match_mask = df_active["seniority_pred_lookup"].notna()
match_rate = match_mask.mean()

v2_labels = set(df_sen_v2["label"].unique())
eval_mask = match_mask & df_active["seniority"].isin(v2_labels)

accuracy_on_matched = (df_active.loc[eval_mask, "seniority_pred_lookup"]
                        == df_active.loc[eval_mask, "seniority"]).mean()

print("Seniority lookup baseline")
print("Match rate (titles found in seniority-v2):", round(match_rate, 3))
print("Accuracy on matched (truth within v2 label set):", round(accuracy_on_matched, 3))
print("Coverage for evaluation (matched & truth in v2):", int(eval_mask.sum()), "/", len(df_active))
```

✓ 0.0s

Seniority lookup baseline
 Match rate (titles found in seniority-v2): 0.228
 Accuracy on matched (truth within v2 label set): 0.793
 Coverage for evaluation (matched & truth in v2): 121 / 623

Department Baseline Results

Applying the same lookup logic to department prediction yields a match rate of 6.6%. On the matched cases, the method achieves an accuracy of 95.1%, covering 41 out of 623 active job entries.

```
df_active["department_pred_lookup"] = df_active["position_norm"].map(lookup_dep)

match_mask = df_active["department_pred_lookup"].notna()
match_rate = match_mask.mean()

# nur evaluieren, wenn truth im v2 label space liegt
v2_labels = set(df_dep_v2["label"].unique())
eval_mask = match_mask & df_active["department"].isin(v2_labels)

accuracy_on_matched = (df_active.loc[eval_mask, "department_pred_lookup"]
                        == df_active.loc[eval_mask, "department"]).mean()

print("Department lookup baseline")
print("Match rate (titles found in department-v2):", round(match_rate, 3))
print("Accuracy on matched (truth within v2 label set):", round(accuracy_on_matched, 3))
print("Coverage for evaluation (matched & truth in v2):", int(eval_mask.sum()), "/", len(df_active))
```

✓ 0.0s

Department lookup baseline
 Match rate (titles found in department-v2): 0.066
 Accuracy on matched (truth within v2 label set): 0.951
 Coverage for evaluation (matched & truth in v2): 41 / 623

Overall, the lookup-based baseline demonstrates strong performance when exact matches are available, but suffers from limited coverage. This limitation motivates subsequent approaches that generalize beyond exact string matching by leveraging richer text representations.

Code Reference: 02_Baseline Seniority.ipynb, 02_Baseline Department.ipynb

5.2 Embedding-based Seniority Classification (Zero-shot)

5.2.1 Embedding-based (Zero-shot) Seniority Classification

As a more flexible alternative to strict rule-based matching, we implemented a zero-shot embedding approach using the Sentence-Transformer model “all-MiniLM-L6-v2”. Each active job title is encoded into a dense vector representation. In parallel, we encode the set of seniority class labels (Junior, Professional, Senior, Lead, Management, and Director) as text strings. For each job title, cosine similarities between the job embedding and all label embeddings are computed. The seniority label with the highest similarity score is then assigned to the job title.

```

job_embeddings = model.encode(df_active["position"].astype(str).tolist())

sim = cosine_similarity(job_embeddings, sen_label_embeddings)
best_idx = sim.argmax(axis=1)

df_active["seniority_pred_embedding"] = [sen_labels[i] for i in best_idx]
df_active[["position", "seniority", "seniority_pred_embedding"]].head(5)

```

✓ 0.5s

	position	seniority	seniority_pred_embedding
0	Prokurist	Management	Professional
1	CFO	Management	Management
2	Betriebswirtin	Professional	Lead
3	Prokuristin	Management	Director
4	CFO	Management	Management

The overall accuracy of this approach is 36.9%, which is relatively low. However, the confusion matrix provides valuable insights into the error structure and shows that many misclassifications occur between adjacent seniority levels.

```

pd.crosstab(
    df_active["seniority"],
    df_active["seniority_pred_embedding"],
    rownames=["truth"],
    colnames=["pred"],
    normalize="index"
).round(3)

```

✓ 0.0s

	pred	Director	Junior	Lead	Management	Professional	Senior
truth	Director	0.941	0.000	0.000	0.029	0.029	0.000
Junior	0.167	0.250	0.083	0.083	0.083	0.333	
Lead	0.064	0.024	0.104	0.640	0.128	0.040	
Management	0.240	0.047	0.026	0.578	0.068	0.042	
Professional	0.079	0.088	0.060	0.551	0.190	0.032	
Senior	0.023	0.023	0.000	0.182	0.091	0.682	

As seniority categories represent an ordered hierarchy, we complement exact accuracy with tolerance-based evaluation metrics that measure whether predictions fall within ± 1 or ± 2 levels.

The model achieves 63.9% accuracy within ± 1 level and 73.2% within ± 2 levels,

```
order = ["Junior", "Professional", "Senior", "Lead", "Management", "Director"]
rank = {k: i for i, k in enumerate(order)}

df_active["truth_rank"] = df_active["seniority"].map(rank)
df_active["pred_rank"] = df_active["seniority_pred_embedding"].map(rank)

# absolute distance in levels
df_active["level_diff"] = (df_active["truth_rank"] - df_active["pred_rank"]).abs()

acc_exact = (df_active["level_diff"] == 0).mean()
acc_pm1 = (df_active["level_diff"] <= 1).mean()
acc_pm2 = (df_active["level_diff"] <= 2).mean()

print("Exact accuracy:", round(acc_exact, 3))
print("Accuracy within  $\pm 1$  level:", round(acc_pm1, 3))
print("Accuracy within  $\pm 2$  levels:", round(acc_pm2, 3))
✓ 0.0s

Exact accuracy: 0.369
Accuracy within  $\pm 1$  level: 0.639
Accuracy within  $\pm 2$  levels: 0.732
```

Code reference: 03_Embedding_Seniority.ipynb

5.2.2 Embedding-based (Zero-shot) Department Classification

Building on the embedding-based seniority classification (Section 5.2.1), we applied the same zero-shot sentence embedding strategy to predict the department of each active job entry. Again, job titles are encoded using the Sentence-Transformer “all-MiniLM-L6-v2” and compared via cosine similarity against embeddings of the predefined department labels (Marketing, Sales, Project Management, IT, Finance, HR, and Other). Each job is assigned the label with the highest similarity score.

An inspection of the predictions reveals a strong dominance of the department label “Other” in the annotated data.

```
df_active[["position", "department", "department_pred_embedding"]].head(10)
```

	position	department	department_pred_embedding
0	Prokurist	Other	HR
1	CFO	Other	Finance
2	Betriebswirtin	Other	IT
3	Prokuristin	Other	HR
4	CFO	Other	Finance
5	Solutions Architect	Information Technology	Project Management
6	Medizintechnik Beratung	Consulting	Marketing
7	Director expansión de negocio.	Business Development	Project Management
8	Gerente comercial	Sales	Marketing
9	Administrador Unico	Administrative	HR

Motivated by this insight, we explored whether historical job information could provide additional context for resolving ambiguous “Other” labels, under the assumption that individuals rarely change their professional domain across roles. For each individual, we inspected previous job entries marked as INACTIVE and extracted the most frequent non-Other department label, if available. This inferred department was then used to replace the Other label of the corresponding active position.

```
from collections import Counter

def dept_from_history(cv):
    past_departments = []
    for job in cv:
        if job["status"] == "INACTIVE" and job["department"] != "Other":
            past_departments.append(job["department"])
    if not past_departments:
        return None
    return Counter(past_departments).most_common(1)[0][0]
```

Applying this refinement affected 27.3% of all active job entries and resolved nearly 49.4% of previously Other labels by assigning a more specific department.

```
changed_rate = (df_active["department"] != df_active["department_fixed"]).mean()

coverage_other = (
    df_active.loc[df_active["department"] == "Other", "department_fixed"]
    .ne("Other")
    .mean()
)

changed_rate, coverage_other

(np.float64(0.27287319422150885), np.float64(0.4941860465116279))
```

When re-evaluating the embedding-based department predictions against the refined labels, overall exact accuracy was 11.6%.

```
embedding_accuracy_history = (
    df_active["department_fixed"] == df_active["department_pred_embedding"]
).mean()

embedding_accuracy_history
✓ 0.0s

np.float64(0.11556982343499198)
```

Finally, when aggregating departments into higher-level functional clusters (e.g., Tech, Commercial, Corporate), the hierarchical accuracy rises to 36.4%, indicating that the embedding-based approach captures broader functional similarities even when fine-grained classification remains challenging.

```
HIGH_LEVEL_MAP = {
    # Tech
    "IT": "Tech",
    "Information Technology": "Tech",

    # Commercial
    "Marketing": "Commercial",
    "Sales": "Commercial",
    "Business Development": "Commercial",
    "Project Management": "Commercial",
    "Consulting": "Commercial",

    # Corporate
    "HR": "Corporate",
    "Human Resources": "Corporate",
    "Finance": "Corporate",
    "Administrative": "Corporate",

    # Other
    "Other": "Other"
}
✓ 0.0s

df_active["department_fixed_cluster"] = df_active["department_fixed"].map(HIGH_LEVEL_MAP)
df_active["department_pred_cluster"] = df_active["department_pred_embedding"].map(HIGH_LEVEL_MAP)
✓ 0.0s

hierarchical_accuracy = (
    df_active["department_fixed_cluster"] ==
    df_active["department_pred_cluster"]
).mean()

hierarchical_accuracy
✓ 0.0s

np.float64(0.36436597110754415)
```

Conclusion:

Department classification from job titles is inherently more challenging than seniority prediction. While seniority levels form a well-defined, ordinal hierarchy that is relatively consistent across organizations and industries, department labels are heterogeneous, weakly standardized, and often context-dependent. Identical job titles may correspond to different departments across firms, and many roles span multiple functional areas. This ambiguity is further amplified by the frequent use of catch-all categories such as Other, which reflect annotation uncertainty rather than semantic meaning.

Code reference: 03_Embedding_Department.ipynb

5.3 TF-IDF + Logistic Regression

5.3.1 TF-IDF + Logistic Regression Seniority

As an interpretable supervised classification approach, we implemented a TF-IDF-based text classification model combined with a multinomial logistic regression classifier. The model is trained on the external seniority-v2 dataset and subsequently applied to active job titles extracted from the LinkedIn CV data.

To train and evaluate the TF-IDF classifier, the seniority-v2 dataset is split into a training and a test set using a stratified 70/30 split. Stratification ensures that the relative class distribution of seniority levels is preserved in both subsets, providing a reliable basis for performance evaluation. (since there is an imbalance:

label	
Senior	3733
Lead	3546
Director	984
Management	756
Junior	409)

```
from sklearn.model_selection import train_test_split

X = df["text"]
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

X_train.shape, X_test.shape
```

```
((6599,), (2829,))
```

Job titles are transformed into TF-IDF representations using unigrams and bigrams to capture both individual keywords and short phrases. Rare and overly frequent terms are filtered to reduce noise.

A multinomial logistic regression classifier with balanced class weights is applied to address class imbalance.

Both steps are combined in a single pipeline to guarantee consistent preprocessing during training and inference.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            ngram_range=(1, 2), # Unigrams + Bigrams
            min_df=2,           # Wörter müssen mind. 2x vorkommen
            max_df=0.9,         # sehr häufige Wörter ignorieren
            lowercase=True
        )
    ),
    (
        "clf",
        LogisticRegression(
            max_iter=1000,
            n_jobs=-1,
            class_weight="balanced" # wichtig wegen Lead/Director
        )
    )
])
```

The trained TF-IDF seniority classifier was subsequently applied to active job titles extracted from the annotated LinkedIn CV dataset. Only job entries marked as ACTIVE were considered, as they represent the individual's professional role.

```
from sklearn.metrics import accuracy_score

mask = df_active["seniority"].notna()

accuracy = accuracy_score(
    df_active.loc[mask, "seniority"],
    df_active.loc[mask, "seniority_pred_tfidf"]
)

accuracy
```

```
0.4333868378812199
```

When evaluated on LinkedIn CV data, the exact accuracy drops to 43.3%, highlighting the domain shift between curated label datasets and real-world job titles.

Problems:

- Label "Professional" does not exist in the training Dataset
- Senior / Lead too high

```
pd.crosstab(
    df_active["seniority"],
    df_active["seniority_pred_tfidf"],
    normalize="index"
).round(3)
```

seniority_pred_tfidf	Director	Junior	Lead	Management	Senior
seniority					
Director	0.853	0.000	0.147	0.000	0.000
Junior	0.000	0.333	0.417	0.000	0.250
Lead	0.000	0.008	0.720	0.032	0.240
Management	0.109	0.000	0.182	0.589	0.120
Professional	0.000	0.032	0.625	0.019	0.324
Senior	0.023	0.045	0.159	0.000	0.773

To address this mismatch, we introduce a confidence-based post-processing step. For predictions initially assigned to Senior or Lead with a low model confidence, the label is reassigned to Professional. This reflects the intuition that ambiguous titles often correspond to mid-level roles rather than clearly senior positions.

As a result, the overall accuracy on active LinkedIn job entries increases from **43.3% to 60.4%**. When accounting for the ordinal nature of seniority levels, the model correctly predicts **82.0% of cases within ± 1 level and 90.7% within ± 2 levels**, indicating that remaining errors are largely confined to neighboring seniority categories.

5.3.2 TF-IDF + Logistic Regression Department

Analogous to the seniority task, we apply a supervised TF-IDF-based text classification model with a multinomial logistic regression classifier to predict the department of active LinkedIn job entries. The model is trained on the external department-v2 dataset and then transferred to active job titles extracted from the Linked in CV data.

Before training, we removed the label "Other" from the department dataset. In our view, "Other" represents a heterogeneous catch-all category rather than a meaningful department class.

Training on such a label would encourage the model to learn non-informative patterns and would reduce interpretability and actionable usefulness of predictions.

```
df_dep = df_dep[df_dep["label"] != "Other"].reset_index(drop=True)

df_dep["label"].value_counts()
```

```
label
Marketing      4295
Sales           3328
Information Technology  1305
Business Development    620
Project Management    201
Consulting         167
Administrative        83
Purchasing          40
Customer Support     33
Human Resources      31
Name: count, dtype: int64
```

When transferring the model to LinkedIn CVs, we observed that department ground truth labels are often set to „Other“ for active jobs. To increase evaluable coverage, we applied a history-based resolution step: for each person, we infer a resolved department label from their employment history by taking the most frequent (mode) department observed in past job entries (excluding missing and „Other“). If the active job’s department label is missing or „Other“, it is replaced by this history-based inferred label. This yields a more consistent reference label for evaluation on the active subset. Using this approach, 180 out of 344 active job entries with originally undefined or Other departments could be resolved, substantially increasing the amount of labeled data available for evaluation.

```
hist = df_all[df_all["department_norm"].notna() & (df_all["department_norm"] != "Other")].copy()

hist_mode = (
    hist.groupby("person_id")["department_norm"]
    .agg(lambda s: s.value_counts().idxmax())
    .to_dict()
)

df_active["department_norm"] = df_active["department"].apply(_norm_dep)

df_active["department_gt_resolved"] = df_active["department_norm"]
needs_fix = df_active["department_gt_resolved"].isna() | (df_active["department_gt_resolved"] == "Other")
df_active.loc[needs_fix, "department_gt_resolved"] = df_active.loc[needs_fix, "person_id"].map(hist_mode)

print("\nResolved GT availability on ACTIVE:")
print(df_active["department_gt_resolved"].value_counts(dropna=False).head(20))
```

Resolved GT availability on ACTIVE:

```
department_gt_resolved
NaN      164
Information Technology    96
Consulting      78
```

```

Sales                69
Project Management   58
Business Development 38
Marketing            37
Administrative        29
Human Resources       26
Purchasing           18
Customer Support     10
Name: count, dtype: int64

```

After resolving missing and Other department labels using historical information, the TF-IDF department model is evaluated on 459 active job entries, achieving an accuracy of 34.4%. This result highlights the increased difficulty of department prediction when transferring the model to noisy, real world LinkedIn profiles.

```

eval_mask = df_active["department_gt_resolved"].notna() & (df_active["department_gt_resolved"] != "Other")

acc_active = accuracy_score(
    df_active.loc[eval_mask, "department_gt_resolved"],
    df_active.loc[eval_mask, "department_pred_model"]
)

print("\nEvaluation on ACTIVE jobs (model vs resolved GT; no 'Other' in eval)")
print(f"Evaluated ACTIVE jobs: {eval_mask.sum()} / {len(df_active)}")
print(f"Accuracy: {acc_active:.4f}")
✓ 0.0s

Evaluation on ACTIVE jobs (model vs resolved GT; no 'Other' in eval)
Evaluated ACTIVE jobs: 459 / 623
Accuracy: 0.3442

```

Code Reference: TF-IDF_logreg_seniority.ipynb, TF-IDF_logreg_department.ipynb

5.4. Features Engineering and Conventional Models

5.4.1 Department Classification

We focus on feature engineering combined with conventional machine learning models. Instead of relying primarily on textual representations, meaningful structured features are derived from the linkedin-cvs-annotated.json, such as career history indicators and categorical job attributes. The features "num_previous_jobs" and "total_years_experience" were introduced to capture an individual's career progression beyond the current job title.

Initially, the dataset including features and label is split into a training and a test set using a 80/20 split. Count encoding was then applied to position and seniority to provide the model with a simple numeric representation of category frequency, helping it handle high-cardinality features without creating sparse vectors. The numeric features num_previous_jobs and total_years_experience were included along with position and seniority to capture career progression and overall experience, which are meaningful signals for predicting the domain. Together, these features give the models both categorical context and quantitative career information in a compact form.

The three models Logistic Regression, Random Forest, and CatBoost, were chosen to train in order to cover a range of modeling approaches suitable for categorical data. These models offer a balanced perspective on model performance before applying more advanced feature engineering.

```
#Define models
random_forest = RandomForestClassifier(n_estimators=200, random_state=42)
CatBoost = CatBoostClassifier(verbose=False, random_seed=42)
Logistic_Regression = LogisticRegression(max_iter=6000)

# Train and evaluate
for model in [random_forest, CatBoost, Logistic_Regression]:
    complete_pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('model', model)])
    complete_pipeline.fit(X_train, y_train_encoded)
    y_pred = complete_pipeline.predict(X_test)
    acc = accuracy_score(y_test_encoded, y_pred)
    print(f"{type(model).__name__} accuracy: {acc:.4f}")

RandomForestClassifier accuracy: 0.4320
CatBoostClassifier accuracy: 0.4720
LogisticRegression accuracy: 0.5360
```

All three models—Logistic Regression, Random Forest, and CatBoost—achieved relatively low accuracy, staying around 40-50%. This indicates that the encoded features did not adequately represent the information needed to distinguish different job domains which highlighted the need for more meaningful feature transformations and motivated the subsequent model improvement steps.

Additional feature derived from file department-v2 (“predicted_department”) and One-hot encoder are incorporated to further improve model performance.

```
preprocessor_rb_dm = ColumnTransformer(
    transformers=[
        ("dept", OneHotEncoder(handle_unknown="ignore"), ["predicted_department"]),
        ("pos", OneHotEncoder(handle_unknown="ignore"), ["seniority"]], remainder="passthrough")
#training
for model in [random_forest, CatBoost, Logistic_Regression]:
    complete_pipeline = Pipeline(steps=[('preprocessor', preprocessor_rb_dm), ('model', model)])
    complete_pipeline.fit(X_train_rb_dm, y_train_rb_dm)
    y_pred_rb_dm = complete_pipeline.predict(X_test_rb_dm)
    acc = accuracy_score(y_test_rb_dm, y_pred_rb_dm)
    print(f"{type(model).__name__} accuracy: {acc:.4f}")

RandomForestClassifier accuracy: 0.6240
CatBoostClassifier accuracy: 0.6080
LogisticRegression accuracy: 0.6640
```

All three models reached an accuracy of more than 60%, indicating that the additional domain knowledge helped capture patterns more effectively. Among the three models, LogisticRegression achieved better performance with highest accuracy of 66%

5.4.2 Seniority Classification

Similarly, new features `num_previous_jobs` and `total_years_experience` derived from the LinkedIn are utilized to predict the Seniority as well as feature position and department (exclusively for seniority prediction). The dataset is then split into a training and a test set using a 80/20 split. In next step, count encoding are also applied to position and department before training with three models Logistic Regression, Random Forest, and CatBoost.

```

categorical_cols = ['position', 'department']
numeric_cols = ['num_previous_jobs', 'total_years_experience']
categorical_transformer = ce.CountEncoder(cols=categorical_cols)
preprocessor = ColumnTransformer(transformers=[('cat', categorical_transformer, categorical_cols)], remainder='passthrough')

for model in [random_forest, CatBoost, LogisticRegression]:
    complete_pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('model', model)])
    complete_pipeline.fit(X_train, y_train)
    y_pred = complete_pipeline.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"{type(model).__name__} accuracy: {acc:.4f}")

... RandomForestClassifier accuracy: 0.4320
CatBoostClassifier accuracy: 0.4480
LogisticRegression accuracy: 0.4000

```

All three models Logistic Regression, Random Forest, and CatBoost achieved relatively low accuracy, staying around 40%. This indicates that the encoded features did not adequately represent the information needed to distinguish different seniority levels. In particular, job titles with similar seniority meanings but different wording were treated as separate categories, which added noise rather than useful signal. As a result, the models showed limited ability to generalize. These findings highlighted the need for more meaningful feature transformations and motivated the subsequent model improvement steps.

To improve the result, additional feature derived from file `seniority-v2` (“`predicted_seniority`”) and One-hot encoder are added.

```

#Encode the features
preprocessor_rb = ColumnTransformer(
    transformers=[("dept", OneHotEncoder(handle_unknown="ignore"), ["department"]),
                  ("pos", OneHotEncoder(handle_unknown="ignore"), ["predicted_seniority"])], remainder="passthrough")

#training
for model in [random_forest, CatBoost, LogisticRegression]:
    complete_pipeline = Pipeline(steps=[('preprocessor', preprocessor_rb), ('model', model)])
    complete_pipeline.fit(X_train_rb, y_train_rb)
    y_pred_rb = complete_pipeline.predict(X_test_rb)
    acc = accuracy_score(y_test_rb, y_pred_rb)
    print(f"{type(model).__name__} accuracy: {acc:.4f}")

... RandomForestClassifier accuracy: 0.6560
CatBoostClassifier accuracy: 0.6800
LogisticRegression accuracy: 0.6480

```

Key Observation: Model performance improves significantly after incorporating rule-based features and categorical encoding, the structured career signals are informative for both tasks.

Predictions remain less reliable for rare categories, suggesting that data imbalance limits the model's ability to generalize across all classes. Models capture high-level patterns well but struggle with fine-grained distinctions inherent in career and domain classification.

Code Reference: 06_Feature_engineering_Seniority.ipynb;
06_Feature_engineering_Domain.ipynb

5.5. Fine-tuning: Domain & Seniority

Analogous to the conventional text classification approaches, we apply a supervised Transformer-based model to predict both the department (domain) and seniority level of active LinkedIn job entries. The approach is based on fine-tuning a distilbert-base-multilingual-cased model, which is well suited to handle the multilingual nature of the dataset, as job titles and descriptions appear in both German and English.

The model is trained on annotated LinkedIn CV data, restricted to ACTIVE job entries that provide complete department and seniority labels. Each job entry is represented as a single text sequence constructed by concatenating relevant job attributes, including the job title, organization name, LinkedIn metadata, employment dates, and job status. This unified text representation serves as the sole input to the classification models. Two separate multi-class classifiers are trained: one for department prediction and one for seniority prediction.

```
# Department label mapping
domain_labels = sorted(df_train["department"].unique().tolist())
domain2id = {lab:i for i, lab in enumerate(domain_labels)}
id2domain = {i:lab for lab, i in domain2id.items()}
df_train["domain_id"] = df_train["department"].map(domain2id)

# Seniority label mapping
sen_labels = sorted(df_train["seniority"].unique().tolist())
sen2id = {lab:i for i, lab in enumerate(sen_labels)}
id2sen = {i:lab for lab, i in sen2id.items()}
df_train["seniority_id"] = df_train["seniority"].map(sen2id)

print("Domain labels:", domain2id)
print("Seniority labels:", sen2id)

Domain labels: {'Administrative': 0, 'Business Development': 1, 'Consulting': 2, 'Customer Support': 3, 'Human Resources': 4, 'Information Techno
Seniority labels: {'Director': 0, 'Junior': 1, 'Lead': 2, 'Management': 3, 'Professional': 4, 'Senior': 5}
```

```

train_df, val_df = train_test_split(
    df_train,
    test_size=0.2,
    random_state=42,
    stratify=df_train["domain_id"]
)

ds_dom_train = Dataset.from_pandas(train_df[["text", "domain_id"]].rename(columns={"domain_id": "label"}))
ds_dom_val = Dataset.from_pandas(val_df[["text", "domain_id"]].rename(columns={"domain_id": "label"}))

ds_sen_train = Dataset.from_pandas(train_df[["text", "seniority_id"]].rename(columns={"seniority_id": "label"}))
ds_sen_val = Dataset.from_pandas(val_df[["text", "seniority_id"]].rename(columns={"seniority_id": "label"}))

print(ds_dom_train, ds_sen_train)

Dataset({
  features: ['text', 'label', '__index_level_0__'],
  num_rows: 498
}) Dataset({
  features: ['text', 'label', '__index_level_0__'],
  num_rows: 498
})

```

Prior to training, categorical department and seniority labels are encoded into numerical identifiers. The labeled dataset is then split into training and validation subsets using a stratified split based on department labels to preserve the original class distribution. Model performance is evaluated using both accuracy and macro F1-score, where the latter is particularly informative in the presence of class imbalance, as it assigns equal importance to all classes regardless of their frequency.

Epoch	Training Loss	Validation Loss	Accuracy	Macro F1
1	No log	1.633456	0.552000	0.064667
2	1.786300	1.612308	0.552000	0.064667
3	1.786300	1.596874	0.552000	0.064667

The fine-tuned department classification model achieves an accuracy of 55.2%, but its macro F1-score of 0.065 indicates substantial difficulty in correctly classifying less frequent department categories. This suggests that the model is heavily influenced by dominant classes and struggles with balanced domain prediction, a challenge that is consistent with the heterogeneous and imbalanced nature of department labels, especially the broad “Other” category.

Epoch	Training Loss	Validation Loss	Accuracy	Macro F1
1	No log	1.480392	0.408000	0.158980
2	1.512100	1.225976	0.544000	0.220506
3	1.512100	1.119938	0.600000	0.343397

In contrast, the seniority classification model performs more robustly, achieving an accuracy of 60.0% and a macro F1-score of 0.343. These results indicate that seniority levels are more consistently encoded in job titles and are therefore easier to infer from short textual inputs compared to department categories.

```
dom_pred_ids = predict_ids(trainer_dom, df_unlab["text"].tolist(), batch_size=64)
sen_pred_ids = predict_ids(trainer_sen, df_unlab["text"].tolist(), batch_size=64)

df_unlab["pred_domain"] = [id2domain[i] for i in dom_pred_ids]
df_unlab["pred_seniority"] = [id2sen[i] for i in sen_pred_ids]

# Save
OUTFILE = "predictions_finetuned_approach3.csv"
df_unlab.to_csv(OUTFILE, index=False)

print("Saved:", OUTFILE)
df_unlab[["position", "organization", "status", "pred_domain", "pred_seniority"]].head(10)
```

Finally, both trained models are applied to the unlabeled LinkedIn CV dataset to generate predicted department and seniority labels for all active job entries. The predicted numerical labels are mapped back to human-readable categories and stored alongside the original job information, enabling automatic enrichment of CV data with inferred career attributes. Overall, this approach demonstrates the effectiveness of Transformer-based fine-tuning for seniority prediction, while also highlighting the limitations of supervised learning for fine-grained department classification under strong class imbalance.

Code Reference: 07_Finetune_Distilbert_CV_Classification.ipynb

5.6 Pseudo-Labeling & Self-Training (TF-IDF)

```

train_gold, val_gold = train_test_split(
    df_gold,
    test_size=0.2,
    random_state=42,
    stratify=df_gold["domain"]
)

print("train_gold:", len(train_gold), "val_gold:", len(val_gold))

train_gold: 2110 val_gold: 528

```

This approach demonstrates that iterative self-training with pseudo-labeling is an effective strategy for improving classification performance when labeled data is limited. Starting from a supervised TF-IDF-based Logistic Regression baseline trained on 2,110 annotated LinkedIn CVs, the model progressively improves by incorporating high-confidence pseudo-labeled samples from a large pool of unlabeled profiles.

A key factor contributing to these improvements is the rich feature construction, which aggregates multiple profile attributes—including job title, organization, headline, summary, skills, job description, industry, and location—into a single textual representation. This provides substantially more contextual information than job titles alone and enables the model to learn more discriminative patterns during self-training.

```

tfidf = TfidfVectorizer(ngram_range=(1,2), min_df=2, max_df=0.9)

X_tr = tfidf.fit_transform(train_gold["text"])
X_va = tfidf.transform(val_gold["text"])

teacher_dom = LogisticRegression(max_iter=3000, class_weight="balanced")
teacher_dom.fit(X_tr, train_gold["domain"])

teacher_sen = LogisticRegression(max_iter=3000, class_weight="balanced")
teacher_sen.fit(X_tr, train_gold["seniority"])

pred_dom_va = teacher_dom.predict(X_va)
pred_sen_va = teacher_sen.predict(X_va)

print("TEACHER DOMAIN accuracy:", accuracy_score(val_gold["domain"], pred_dom_va))
print("TEACHER DOMAIN macro-F1:", f1_score(val_gold["domain"], pred_dom_va, average="macro"))
print("\nTEACHER DOMAIN report:\n", classification_report(val_gold["domain"], pred_dom_va))

print("TEACHER SENIORITY accuracy:", accuracy_score(val_gold["seniority"], pred_sen_va))
print("TEACHER SENIORITY macro-F1:", f1_score(val_gold["seniority"], pred_sen_va, average="macro"))
print("\nTEACHER SENIORITY report:\n", classification_report(val_gold["seniority"], pred_sen_va))

```


TEACHER DOMAIN accuracy: 0.5738636363636364

TEACHER DOMAIN macro-F1: 0.516029049517747

TEACHER DOMAIN report:

	precision	recall	f1-score	support
Administrative	0.10	0.24	0.14	17
Business Development	0.53	0.62	0.57	16
Consulting	0.68	0.59	0.63	39
Customer Support	0.38	0.60	0.46	10
Human Resources	0.39	0.64	0.49	14
Information Technology	0.67	0.71	0.69	62
Marketing	0.33	0.63	0.43	27
Other	0.80	0.53	0.63	250
Project Management	0.54	0.63	0.58	35
Purchasing	0.30	0.64	0.41	14
Sales	0.69	0.61	0.65	44
accuracy			0.57	528
macro avg	0.49	0.59	0.52	528
weighted avg	0.66	0.57	0.60	528

Empirically, the results show consistent performance gains across iterations, particularly for domain classification, which is the more challenging task. Domain accuracy improves from 0.574 at initialization to 0.640 after five self-training iterations, while the macro F1-score increases from 0.516 to 0.554. These gains indicate that the pseudo-labeling strategy successfully introduces informative training examples and improves performance across minority domain classes, not merely the dominant ones.

TEACHER SENIORITY accuracy: 0.7746212121212122

TEACHER SENIORITY macro-F1: 0.7747358024641627

TEACHER SENIORITY report:

	precision	recall	f1-score	support
Director	0.75	0.87	0.81	31
Junior	0.61	0.84	0.70	44
Lead	0.72	0.63	0.67	100
Management	0.69	0.78	0.73	69
Professional	0.85	0.78	0.82	251
Senior	0.89	0.94	0.91	33
accuracy			0.77	528
macro avg	0.75	0.81	0.77	528
weighted avg	0.78	0.77	0.78	528

In contrast, seniority prediction consistently outperforms domain prediction throughout the process. The seniority model starts with a relatively strong baseline performance of 0.775 accuracy and 0.775 macro F1-score, and improves modestly to 0.784 accuracy and 0.788 macro F1-score after self-training. This suggests that seniority levels are more directly encoded in textual cues such as role titles and descriptions, making them easier to infer even without extensive data augmentation.

Training history:						
	iter	train_size	dom_acc	dom_macroF1	sen_acc	sen_macroF1
0	1	2110	0.573864	0.516029	0.774621	0.774736
1	2	2303	0.596591	0.528000	0.774621	0.776984
2	3	2548	0.611742	0.538896	0.780303	0.781592
3	4	2820	0.625000	0.543623	0.782197	0.787208
4	5	3110	0.640152	0.554149	0.784091	0.788312

The size of the training dataset grows steadily from 2,110 samples to 3,110 samples over five iterations, and this increase correlates positively with performance improvements. Importantly, no degradation is observed in later iterations, indicating that confidence-based filtering, class-balanced sampling, and exclusion of the heterogeneous “Other” category effectively prevent error propagation. Overall, the results show that iterative self-training is particularly beneficial for complex and ambiguous tasks such as domain classification, while providing stable and incremental gains for more structurally defined tasks like seniority prediction.

Code Reference: 08_Pseudo-Labeling_CV_Domain_Seniority.ipynb

5.7 LLM-Based Classification (Google Gemini API)

5.7.1 Seniority

As a complementary and exploratory approach, we implemented a large language model (LLM)-based seniority classifier using Google Gemini. This approach represents our own creative solution beyond conventional rule-based and supervised machine learning methods.

Setup and Prompting Strategy

The Gemini API is accessed via an API key stored securely in a local .env file and loaded at runtime, ensuring that sensitive credentials are not hard-coded or exposed in the codebase.

For each active LinkedIn job entry, the model is prompted with a textual description consisting of:

- The job title
- The company name

```
prompt = f"""Job title: {row["position"]}
Company: {row["organization"]}
Which seniority label applies?
"""
```

To ensure consistent outputs, we define explicit system instructions that constrain the model to return exactly one label from a predefined, closed set of seniority categories:

```
SENIORITY_LABELS = [
    "Junior",
    "Professional",
    "Senior",
    "Lead",
    "Management",
    "Director"
]

SYSTEM = (
    "You are a classifier. "
    f"Return exactly ONE label from: {' '.join(SENIORITY_LABELS)}. "
    "Output ONLY the label."
)
```

Each active job is processed independently. The model generates a single seniority prediction per job, which is then stored and evaluated against the annotated seniority labels in the LinkedIn CV dataset.

```
from tqdm.auto import tqdm
from sklearn.metrics import accuracy_score

preds = []

for _, r in tqdm(df_active.iterrows(), total=len(df_active)):
    prompt = f"""Job title: {r["position"]}
Company: {r["organization"]}
Which seniority label applies best? Choose from: {' '.join(SENIORITY_LABELS)}
"""
    resp = client.models.generate_content(
        model=MODEL,
        contents=prompt,
        config={"system_instruction": SYSTEM}
    )
    preds.append(resp.text.strip())

df_active["seniority_pred"] = preds

acc = accuracy_score(df_active["seniority"], df_active["seniority_pred"])
print("Accuracy:", round(acc, 4))

/Users/lennartredlich/Library/Python/3.9/lib/python/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found
from .autonotebook import tqdm as notebook_tqdm
100%|██████████| 623/623 [04:41<00:00, 2.21it/s]
Accuracy: 0.549
```


Evaluation:

On the active job subset, the Gemini-based classifier achieves:

- Exact Accuracy: 54.9%
- Ordinal Accuracy (± 1 level): 82.5%

The confusion matrix shows that most errors occur between adjacent seniority levels, indicating that the model captures the hierarchical structure of seniority reasonably well, even when exact classification is challenging.

```
cm = pd.crosstab(
    df_active["seniority"],
    df_active["seniority_pred"],
    normalize="index"
).round(3)

cm
```

seniority_pred \ seniority	Director	Junior	Lead	Management	Professional	Senior
Director	1.000	0.000	0.000	0.000	0.000	0.000
Junior	0.000	0.583	0.000	0.000	0.333	0.083
Lead	0.248	0.008	0.216	0.312	0.208	0.008
Management	0.557	0.000	0.000	0.422	0.010	0.010
Professional	0.014	0.056	0.000	0.157	0.741	0.032
Senior	0.068	0.000	0.000	0.136	0.045	0.750

5.7.2 Department

As an additional, exploratory approach, we implemented a department classifier using Google Gemini. Analogous to the LLM-based seniority classification described above, this approach evaluates whether a large language model can infer professional departments from short and noisy LinkedIn job metadata.

The technical setup (API access via a .env file, system instructions, and constrained label output) follows the same configuration as in the seniority experiment and is therefore not repeated here.

Data Preparation and „Resolved“ Ground Truth

For evaluation on active jobs, ground-truth department labels are often missing or set to „Other“. To increase evaluable coverage, we apply the same history-based resolution idea as before:

For each person, we infer the most frequent department from their past job entries (excluding missing values and „Other“) and use it to replace „Other“ department labels in active jobs. This produces a more consistent reference label **department_gt_resolved**.

```
df_active["department_gt_resolved"] = df_active["department_norm"]
needs_fix = df_active["department_gt_resolved"].isna() | (df_active["department_gt_resolved"] == "Other")
df_active.loc[needs_fix, "department_gt_resolved"] = df_active.loc[needs_fix, "person_id"].map(hist_mode)

print("\nResolved GT availability on ACTIVE:")
print(df_active["department_gt_resolved"].value_counts(dropna=False).head(20))
```

Resolved GT availability on ACTIVE:

department_gt_resolved	
NaN	164
Information Technology	96
Consulting	78
Sales	69
Project Management	58
Business Development	38
Marketing	37
Administrative	29
Human Resources	26
Purchasing	18
Customer Support	10

Name: count, dtype: int64

NaN: 164 means here: no resolvable history label available.

Prompting and Label Constraints

We define a closed set of 10 department labels. Gemini is guided via a strict system instruction that forces the model to output exactly one label from this set and prohibits free-form text.

Each prediction uses a short prompt containing:

- Job title
- Company name
- The instruction to choose one label from the predefined list

```
DEPARTMENT_LABELS = [
    "Information Technology",
    "Consulting",
    "Sales",
    "Project Management",
    "Business Development",
    "Marketing",
    "Administrative",
    "Human Resources",
    "Purchasing",
    "Customer Support"
]

SYSTEM = (
    "You are a classifier. "
    f"Return exactly ONE label from: {' '.join(DEPARTMENT_LABELS)}. "
    "Output ONLY the label."
)
```

Evaluation Results

On the subset of active jobs with resolved ground truth (n=459), the Gemini department classifier achieves an **accuracy of 61.2%**.

The normalized confusion matrix provides additional insights, showing that misclassifications frequently occur between semantically close departments (eg., Business Development vs. Consulting or Marketing).

```
from tqdm.auto import tqdm
from sklearn.metrics import accuracy_score

preds = []

for _, r in tqdm(df_active_fix.iterrows(), total=len(df_active_fix)):
    prompt = f"""Job title: {r["position"]}
Company: {r["organization"]}
Which department label applies best? Choose from: {", ".join(DEPARTMENT_LABELS)}
"""
    resp = client.models.generate_content(
        model=MODEL,
        contents=prompt,
        config={"system_instruction": SYSTEM}
    )
    preds.append(resp.text.strip())

df_active_fix["department_pred"] = preds

acc = accuracy_score(df_active_fix["department_gt_resolved"], df_active_fix["department_pred"])
print("Accuracy:", round(acc, 4))
```

```
100%|██████████| 459/459 [03:40<00:00, 2.08it/s]
Accuracy: 0.6122
```

department_predicted	Administrative	Business Development	Consulting	Customer Support	Human Resources	Information Technology	Marketing	Project Management	Purchasing	Sales
department_ground_truth										
Administrative	0.724	0.103	0.034	0.034	0.000	0.034	0.000	0.034	0.000	0.034
Business Development	0.000	0.605	0.342	0.000	0.000	0.000	0.000	0.026	0.000	0.026
Consulting	0.077	0.205	0.538	0.000	0.064	0.051	0.013	0.026	0.000	0.026
Customer Support	0.200	0.100	0.000	0.600	0.000	0.100	0.000	0.000	0.000	0.000
Human Resources	0.038	0.077	0.038	0.000	0.654	0.000	0.115	0.038	0.000	0.038
Information Technology	0.052	0.115	0.052	0.010	0.000	0.646	0.000	0.094	0.031	0.000
Marketing	0.081	0.243	0.081	0.000	0.000	0.108	0.459	0.000	0.000	0.027
Project Management	0.034	0.172	0.017	0.000	0.017	0.034	0.034	0.638	0.034	0.017
Purchasing	0.111	0.111	0.000	0.056	0.000	0.000	0.000	0.000	0.722	0.000
Sales	0.072	0.232	0.014	0.000	0.000	0.000	0.029	0.029	0.000	0.623

Conclusion:

Advantages:

- No training data required (zero-shot)
- Strong semantic understanding of job titles
- Best performance on department prediction

Disadvantages:

- API costs (~\$0.001 per prediction)
- Latency (~200-500ms per request)
- Requires internet connectivity

Code Reference: 05_API_Seniority.ipynb, 05_API_Department.ipynb

6. Performance Comparison

The following table summarizes the accuracy of all approaches on the SnapAddy evaluation set (623 ACTIVE jobs):

Approach	Seniority	Department
Baseline (Rules)	--	--
Embedding-based	36.9%	36.4%
Features Engineering	44.8%	53.6%
Fine-tuning	60%	55.2%,
Pseudo-Labeling	64%	78.7%
TF-IDF + LogReg	60.4%	34.4%
LLM API (Gemini)	54.9%	48.4%

Key Findings:

- **Approaches suggested for Seniority:** TF-IDF + Logistic Regression, Pseudo-Labeling and Fine-tuning.
- **Approaches suggested for Department:** Pseudo-Labeling, Fine-tuning, LLM API(though still challenging due to 'Other' category ambiguity)
- **Baseline vs. Advanced:** Rule-based methods perform poorly compared to advanced methods and not be able to generalize without exact string matching. The match rate for seniority is 22.8% with 79% accuracy and for department is only 6.6% with 95.1% accuracy, which show that it only cover small amount of 623 active job entries.
- **Ordinal Performance:** Seniority models achieve ~82% accuracy within ± 1 level (TF-IDF + LogReg approach), indicating they capture hierarchical structure even when exact predictions fail

7. Model Failures & Limitations

7.1 Seniority Prediction Challenges

Semantic Overlap: The boundaries between Lead, Management, and Director are often fuzzy. Job titles like 'Head of Engineering' could reasonably map to any of these categories depending on company size and structure.

Example Failures:

- 'Team Lead' → Predicted as Management (true label: Lead)
- 'Senior Consultant' → Predicted as Senior (true label: Professional)
- 'Junior Developer' → Predicted as Professional (true label: Junior)

7.2 Department Prediction Challenges

The 'Other' Problem: 27% of jobs are labeled as 'Other', which acts as a catch-all for non-standard roles. This creates ambiguity in both training and evaluation.

Multi-Domain Roles: Many modern job titles span multiple departments (e.g., 'Business Development Manager' could be Sales or Business Development).

Example Failures:

- 'Account Manager' → Predicted as Sales (true label: Business Development)
- 'Product Manager' → Predicted as Project Management (true label: Other)
- 'CEO' → Predicted as Other (true label varies by company)

8. Generative AI Usage Documentation

This section documents the use of generative AI tools throughout the project, as required by the course guidelines.

8.1 Tools Used

- **GitHub Copilot:** Autocomplete suggestions during coding
- **ChatGPT:** Occasional syntax lookups and conceptual questions

8.2 Use Cases & Examples

8.2.1 Code Generation & Debugging

Example Prompts:

- "How do I implement TF-IDF with sklearn for multi-class classification with balanced class weights?"
- "Write a Streamlit app with file upload, text search filter, and dataframe display"
- "Debug this pandas error: 'KeyError: status' when filtering ACTIVE jobs"
- "Generate a confusion matrix with normalized values using sklearn"

8.2.2 API Integration

Example Prompts:

- "How do I call the Google Gemini API with system instructions and parse the response?"
- "Write a prompt template for zero-shot classification with a constrained label set"

8.2.3 Data Analysis

Example Prompts:

- "Calculate ordinal accuracy (within ± 1 level) for hierarchical labels"
- "How do I handle missing values in pandas when filtering by department?"

8.2.4 Documentation & Report Writing

Example Prompts:

- "Explain TF-IDF methodology in accessible language for non-technical readers"
- "Please rephrase the following paragraph in a more formal and academic writing style while preserving the original meaning."

8.3 Reflection on AI Assistance

What worked well:

- Rapid prototyping of code scaffolding (e.g., Streamlit app structure)

- Debugging cryptic error messages and pandas quirks
- Generating boilerplate documentation and explanatory text
- Learning new libraries (e.g., Google Generative AI SDK) via examples

Challenges encountered:

- Generated code sometimes lacked context-specific error handling
- AI suggestions occasionally introduced subtle bugs (e.g., incorrect DataFrame indexing)
- Required manual verification of all statistical claims and methodology descriptions
- Prompt engineering took multiple iterations to get desired output quality

9. Conclusion

This project successfully developed a multi-approach machine learning pipeline for predicting professional domain and seniority level from LinkedIn profiles. We implemented and evaluated four distinct methodologies, ranging from simple rule-based baselines to advanced LLM-powered APIs, and deployed a functional Streamlit dashboard demonstrating real-world applicability.

Key Takeaways:

- **Seniority Prediction:** TF-IDF + Logistic Regression achieved 60% exact accuracy and 82% ordinal accuracy (± 1 level), demonstrating that classical ML can effectively capture hierarchical structure in job titles
- **Department Prediction:** LLM APIs, Pseudo-Labeling, Fine-tuning outperformed traditional methods, highlighting the value of large language models for ambiguous, context-dependent classification tasks.
- **Practical Deployment:** The Streamlit dashboard successfully translates model predictions into actionable business intelligence for recruiting and sales use cases

Despite achieving respectable performance, significant room for improvement remains. Future work should focus on feature engineering from career trajectories, ensemble methods combining multiple models, and explainability tools to increase user trust. The 'Other' department category represents a fundamental data quality challenge that would benefit from label refinement or hierarchical classification strategies.

This project demonstrates that automated career profile classification is feasible and valuable, while also highlighting the complexities inherent in modeling human career paths through structured data.

10. Role of each Group Members

This report was collaboratively developed by group members. Each member contributed to data analysis, feature engineering, model development, experimentation, and the interpretation of results. The different tasks and approaches explored in this project were implemented by the group members and are listed in the following section:

Lennart Redlich:

- 01_EDA.ipynb
- 02_Baseline Department.ipynb / 02_Baseline Seniority
- 03_Embedding_Department.ipynb / 03_Embedding_Seniority.ipynb
- 04_TF-IDF_logreg_seniority.ipynb / 04_TF-IDF_logreg_department.ipynb
- 05_API_Department / 05_API_Seniority
- Streamlit Dashboard

Anh Pham

- Feature engineering and conventional machine learning (06_Feature_engineering_Domain.ipynb/ 06_Feature_engineering_Seniority.ipynb)
- Rule-based matching (baseline) for seniority and domains
- Simple interpretable baseline: bag-of-words and TF-IDF + logistic regression classifier for domain or seniority
- Performance Comparison

Van Anh Nguyen

- 4. Data & Exploratory Data Analysis: Preparation of the data analysis and interpretation section based on exploratory results.
- 07_Finetune_Distilbert_CV_Classification.ipynb: Implementation and fine-tuning of a DistilBERT model for CV classification, including preprocessing, training, and evaluation.
- 08_Pseudo-Labeling_CV_Domain_Seniority.ipynb: Development of a pseudo-labeling approach to incorporate unlabeled CV data for domain and seniority classification.

11. Appendix

11.1 Code Structure

Project Directory:

Capstone-Project-/

```

├── files/                                # Data files
│   ├── department-v2.csv
│   ├── linkedin-cvs-annotated.json # Main evaluation dataset
│   ├── linkedin-cvs-not-annotated.json
│   ├── seniority-v2.csv
│   └── snapAddy_Logo.png              # Logo
├── src/
│   ├── notebooks/                      # Analysis notebooks
│   │   ├── 01_EDA.ipynb
│   │   ├── 02_Baseline Department.ipynb
│   │   ├── 02_Baseline Seniority.ipynb
│   │   ├── 03_Embedding_Department.ipynb
│   │   ├── 03_Embedding_Seniority.ipynb
│   │   ├── 04_TF-IDF_logreg_department.ipynb
│   │   ├── 04_TF-IDF_logreg_seniority.ipynb
│   │   ├── 05_API_Seniority.ipynb
│   │   ├── 05_API_Department.ipynb
│   │   ├── 06_Feature_engineering_Domain.ipynb
│   │   ├── 06_Feature_engineering_Seniority.ipynb
│   │   ├── 07_Finetune_Distilbert_CV_Classification.ipynb
│   │   ├── 08_Pseudo-Labeling_CV_Domain_Seniority.ipynb
│   │   └── *.joblib                  # Saved models
├── app.py                             #Streamlit dashboard
├── requirements.txt
└── README.md

```

11.2 Required Python Packages

- pandas, numpy: Data manipulation
- scikit-learn: TF-IDF, Logistic Regression, metrics

- sentence-transformers: Embeddings
- google-generativeai: Gemini API
- streamlit: Web dashboard
- joblib: Model persistence

11.3 GitHub Repository

Full code, notebooks, and data available at:

<https://github.com/LennartRedlich/Capstone-Project->